Deep Research Agent: Landscape Report & Architectural Analysis

- Deep Research Agent: Landscape Report & Architectural Analysis
  - Table of Contents
  - 1. Executive Summary
  - 2. Key Architectural Patterns
  - 3. Tech Stack Components
  - 4. Comparative Analysis: Pros & Cons
  - 5. State-of-the-Art Findings (June 2026)
  - 6. Gaps in the Current Landscape
  - 7. Recommendations
  - 8. Source Index
  - Appendix: Your DRA Design Validation

Deep Research Agent: Landscape Report & Architectural Analysis

Compiled: 2026-06-10
Sources: GitHub (20+ repos), YouTube (47+ videos/talks), Arxiv (40+ papers)
Scope: Architectures, tech stacks, design choices, pros/cons, and recommendations for building a production-grade Deep Research Agent system.

Table of Contents

1. Executive Summary
2. Key Architectural Patterns
3. Tech Stack Components
4. Comparative Analysis: Pros & Cons
5. State-of-the-Art Findings (June 2026)
6. Gaps in the Current Landscape
7. Recommendations
8. Source Index

---

1. Executive Summary

The Deep Research Agent space has exploded in 2025-2026. There are now **20+ significant open-source implementations** (from 172 to 35K+ stars), **40+ academic papers** directly addressing research agents, and a rich ecosystem of tutorials and framework-level tooling.

The field converges around a core insight: the hard problem is not generating research reports – it's *verifying* them. Most systems (GPT-Researcher, STORM, LangChain Open Deep Research) produce plausible output but lack rigorous claim verification. The academic frontier (Search-R1, VERITAS, Self-RAG) is pushing toward RL-trained search, process-level rewards, and adaptive retrieval – but almost no deployed system combines independent verification, calibrated confidence, and adaptive retrieval in a single loop.

Key Takeaways

- **Architecture:** The field is settling on plan-decompose-search-synthesize as the base loop, with verification as the differentiator
- **Training vs. Prompting:** RL-trained agents (Search-R1 family) outperform prompt-engineered ones on benchmarks, but require significant GPU resources
- **Verification:** Independent model-diverse judges outperform homogeneous panels; self-evaluation provides zero calibration information without external labels
- **Retrieval:** Adaptive retrieval (triggered by uncertainty/verification failure) is strictly superior to fixed retrieval strategies
- **Evaluation:** Best current systems achieve only ~19.9% on structural verification tasks – this is an unsolved problem

---

2. Key Architectural Patterns

Pattern A: Recursive Tree Exploration

How it works: Breadth-first recursive search with configurable depth and width. Each iteration generates sub-queries, processes results to extract learnings and new directions, recurses until depth=0.

Examples: dzhng/deep-research (19.1K stars), GPT-Researcher Deep Mode

Flow:

```
Query → Generate sub-queries → Search each → Extract learnings
  ↓
New directions discovered? → Recurse (depth-1) → Compile final report
```

Characteristics: - Simplest architecture (<500 LoC in minimal form) - Configurable breadth (3-10) and depth (1-5) - No explicit verification step - Relies on iterative refinement over correctness checking

---

Pattern B: Plan-and-Execute with Specialized Agents

How it works: A planner decomposes the query into sub-tasks. Specialized agents (searcher, analyzer, writer) execute in parallel or sequence. A publisher/synthesizer aggregates results.

Examples: GPT-Researcher (27.6K stars), LangChain Open Deep Research (11.7K stars), AWS Strands 5-agent framework

Flow:

```
Query → Planner (decompose into sub-questions)
  ↓
Parallel Execution: Agent₁(search q1) | Agent₂(search q2) | Agent₃(search q3)
  ↓
Aggregator/Editor → Structured Report
```

Characteristics: - Most production-deployed pattern - Different models for different roles (cheap for search/summarization, expensive for synthesis) - Parallelizable for latency reduction - No per-claim verification – relies on multi-source aggregation to reduce bias

---

Pattern C: Multi-Perspective Simulation (STORM)

How it works: Discovers diverse expert perspectives on a topic, simulates conversations between perspectives grounded in retrieved sources, curates into outline, then generates cited article.

Examples: Stanford STORM (28.3K stars), Co-STORM

Flow:

```
Topic → Discover Perspectives (N experts)
  ↓
Simulate Grounded Conversations (perspective × questions)
  ↓
Curate Knowledge → Generate Outline → Write Cited Article → Polish
```

Characteristics: - Multi-perspective ensures coverage breadth - 4-module pipeline (Knowledge Curation, Outline, Article, Polish) - Published at NAACL 2024 / EMNLP 2024 - No explicit verification – relies on multi-source grounding - Best for Wikipedia-style comprehensive coverage

---

Pattern D: RL-Trained Search Agent (Search-R1 Family)

How it works: Train the model itself via reinforcement learning to decide when/what to search during step-by-step reasoning. The model generates special tokens to invoke search, then continues reasoning with retrieved results.

Examples: Search-R1 (4.9K stars), R1-Searcher, Alibaba DeepResearch (19.4K stars), WebThinker (1.5K stars), DeepDiver

Flow:

```
Query → Model generates reasoning tokens
  ↓ (model decides to search)
<search>generated query</search> → Retrieval system returns results
  ↓
Model continues reasoning with results → May search again → Final answer
```

Training:

```
SFT warmup (optional) → RL with GRPO/PPO → Outcome reward (correctness)
  + Process rewards (faithfulness, step quality)
```

Characteristics: - SOTA on benchmarks (BrowseComp, FRAMES, HLE) - 7B models can match GPT-4o-mini performance - Requires significant GPU resources for training (8xA100+) - Most principled but least accessible - “Search Intensity Scaling” emerges – model escalates search frequency on harder problems

---

Pattern E: Self-Critique with Reflection Tokens (Self-RAG)

How it works: The model generates special reflection tokens during generation: `[Retrieval]/[No Retrieval]` (decides when to retrieve), `[Relevant]` (relevance check), `[Fully supported]` (grounding check), `[Utility]` (quality). Segment-wise beam search over these critique dimensions.

Examples: Self-RAG (2.4K stars, ICLR 2024 Oral), FLARE (CMU)

Flow:

```
Generate segment → [Retrieval] token?
  → Yes: Retrieve → [Relevant]? → Generate with evidence → [Fully Supported]?
  → No: Continue generation
  ↓
Segment-wise beam search over (relevance × support × utility)
```

Characteristics: - Most elegant adaptive retrieval with learned verification - On-demand retrieval (not always-retrieve) - Configurable inference weights for different dimensions - Requires full model training (Llama-2 base) - ICLR 2024 Oral (top 1% of submissions)

Pattern F: Adversarial Multi-Agent Verification

How it works: Separate agent roles (Builder/Proposer, Skeptic/Verifier, Arbiter/Judge) with deliberately different models to prevent "theatrical disagreement." Claims are structured as a DAG with epistemic layers.

Examples: MARs (Builder/Skeptic/Arbiter), PROClaim (Plaintiff/Defense/Judge), SAGE (Challenger/Planner/Solver/Critic)

Flow:

```
Builder proposes claims with evidence
  ↓
Skeptic (different model) attempts to refute each claim
  ↓
Arbiter adjudicates: Supported | Contested | Insufficient
  ↓
Rejected claims → limitations section (not deleted)
Insufficient claims → trigger more retrieval
```

Characteristics: - Most rigorous for scientific methodology - Model independence prevents correlated errors - Computationally expensive (multiple expensive model calls per claim) - Small community adoption - Best suited for high-stakes research where correctness >> speed

---

Pattern G: Iterative Verify-then-Retrieve Loop

How it works: Generate claims with citations, independently verify each claim against its evidence, route by verification verdict (supported → accept, insufficient → re-search, refuted → revise, weak → switch tool). Loop until convergence.

Examples: Your DRA design (proposed), AlignRAG (NeurIPS 2025), ADORE (enterprise)

Flow:

```
Generate claims + citation links → Independent Verifier scores each
  ↓
Verdict routing:
  supported (≥τ) → accept to final report
  insufficient → re-search with refined query
  refuted → revise claim or mark as contested
  weak → switch retrieval tool
  ↓
Loop until: all claims resolved | max_iters | no-progress | budget exhausted
  ↓
Compile ResearchReport (supported + contested + insufficient claims)
```

Characteristics: - Verification drives retrieval (not the other way around) - Naturally produces per-claim confidence - Separates generation from verification (model independence) - Most architecturally novel – few implementations exist - Requires careful convergence criteria

---

3. Tech Stack Components

3.1 Orchestration Frameworks

Framework	Stars	Type	Best For	Status
LangGraph	11.7K (Open DR)	Graph-based state machines	Complex branching, conditional loops, parallel processing	Active, dominant
CrewAI	~15K	Role-based multi-agent	Quick prototyping, role specialization	Active
AutoGen (Microsoft)	58.8K	Message-passing agents	Distributed, event-driven systems	**Maintenance mode** (deprecated for MAF)
DSPy	~15K	Declarative prompt programming	Optimization, modular pipelines	Active (used by STORM)
veRL	~5K	RL training framework	Training search-augmented agents	Active (Search-R1 family)
AWS Strands	New	Agent framework	AWS Bedrock integration	Active, enterprise

Verdict: LangGraph dominates for deployed research agents. DSPy for modular pipeline optimization. veRL/OpenRLHF for training-based approaches.

3.2 LLM Providers & Models

Role	Recommended Models	Why
Generator (report writing)	Claude Opus/Sonnet, GPT-4.1, Qwen3-235B	Best long-form quality
Planner (query decomposition)	GPT-4.1-mini, Claude Haiku, Qwen3-8B	Fast, cheap, structured output
In-loop Verifier	Different family from generator (e.g., if generator=Claude, verifier=GPT or Gemini)	Model independence prevents correlated errors
Judge Panel	Mix of families: Claude + GPT + Gemini + open-source	Diversity > homogeneity (validated by research)
Reasoning-heavy	o3/o4-mini, DeepSeek-R1, QwQ-32B	Extended reasoning chains
Local/Cost-efficient	Qwen3-8B, LLaMA-3-8B, Mistral	Development/testing

Key insight from literature: “Nine Judges, Two Effective Votes” (arXiv 2605.29800) proves that using 9 LLM judges from correlated families effectively provides only ~2 independent votes. **Model-family diversity is non-**

negotiable for judge panels.

3.3 Search/Retrieval APIs

Tool	Type	Strengths	Weaknesses	Cost
Tavily	AI-native search API	Most popular in agent ecosystem; structured output; fast	Limited depth per result	$0.01/search
Exa	Neural/semantic search	Semantic understanding; finds conceptually related content; citation-native	Smaller index than Google	Free tier available
SearXNG	Meta-search (self-hosted)	Privacy; no API costs; aggregates 70+ engines	Requires hosting; rate limits from upstream	Free (self-host)
Jina Reader	Page extraction	Clean text from any URL; handles JS-rendered pages	Rate limited on free tier	Free tier + paid
Firecrawl	Web scraping + search	Deep extraction; structured data	Rate limits	Free tier + paid
Serper	Google SERP API	Google-quality results	$50/5000 searches	Paid
Semantic Scholar	Academic papers	Best for scientific literature; free	Limited to academic content	Free
Brave Search	Web search API	Good privacy; decent quality	Smaller index	Free tier

Verdict: Tavily is the default for general web research. Exa for semantic/conceptual search. Semantic Scholar for academic papers. Multi-tool strategy is ideal (route by query type).

3.4 Vector Databases (for RAG/Private Data)

Database	Strengths	Best For
Milvus/Zilliz	Scalable, cloud-native	Enterprise private data research
Qdrant	Rust-native performance	High-throughput retrieval
ChromaDB	Simple, embedded	Prototyping, small datasets
FAISS	Facebook, CPU/GPU	Training pipelines, offline indexing
Pinecone	Managed, serverless	Production without ops overhead

3.5 Evaluation & Observability

Tool	Purpose	Notes
LangSmith	LLM tracing, evaluation	LangChain ecosystem native
Weights & Biases	Experiment tracking	Standard for RL training runs
Braintrust	LLM evaluation platform	Purpose-built for LLM-as-judge
Verdict (arXiv 2502.18018)	Judge-time compute scaling	Open-source; applies inference scaling to judges
DeepEval	LLM evaluation framework	Multiple metrics, easy integration

3.6 Deployment Infrastructure

Component	Options
Runtime	FastAPI, Docker, LangGraph Platform, AWS Lambda
Compute	AWS Bedrock, local GPU (NVIDIA), cloud VMs
Frontend	Next.js, Streamlit, Chainlit, Gradio
CI/CD	Standard (GitHub Actions), LangGraph Cloud

---

4. Comparative Analysis: Pros & Cons

Architecture Comparison Matrix

Dimension	Recursive Tree (A)	Plan-Execute (B)	STORM (C)	RL-Trained (D)	Self-RAG (E)	Adversarial (F)	Verify-Loop (G)
Simplicity	5	4	3	1	2	2	3
Verification rigor	1	1	2	3	4	5	5
Confidence scoring	1	1	1	2	3	3	5
Adaptive retrieval	2	2	1	5	5	3	5
Production readiness	4	5	4	2	3	2	3
Cost efficiency	4	3	3	2	3	1	2
Academic grounding	1	2	5	5	5	4	4
Community/ecosystem	4	5	4	3	3	1	1

(Scale: 1=weakest, 5=strongest)

Detailed Pros/Cons by Pattern

Pattern A: Recursive Tree Exploration

Pros	Cons
Extremely simple to implement (<500 LoC)	No verification – hallucinations propagate
Easy to understand and debug	No confidence scoring
Configurable depth/breadth trade-off	Depth explosion on broad topics
Fast iteration for prototyping	Quality ceiling – refinement ≠ correctness

Pattern B: Plan-and-Execute (Multi-Agent)

Pros	Cons
Most battle-tested in production	Multi-source aggregation ≠ verification
Parallelizable (fast wall-clock time)	Agent coordination overhead
Role specialization (use cheap models where possible)	Error propagation across agents
Extensive tooling (LangGraph, GPT-Researcher)	Debugging multi-agent workflows is hard
~$0.40/run for deep research	No per-claim confidence

Pattern C: Multi-Perspective Simulation (STORM)

Pros	Cons
Excellent topic coverage (multi-perspective)	No explicit fact-checking
Academic rigor (NAACL 2024)	Cannot produce publication-ready output
Great for Wikipedia-style articles	Source bias transfers to output
Modular 4-stage pipeline	Over-association of unrelated facts
DSPy-based (optimizable)	Static perspectives (doesn't adapt mid-research)

Pattern D: RL-Trained Search Agent

Pros	Cons
SOTA benchmarks (BrowseComp, FRAMES, HLE)	Requires 8xA100+ for training
Model learns optimal search behavior	Not accessible without GPU infrastructure
Emergent "Search Intensity Scaling"	Static training corpus (Wikipedia)
7B models match GPT-4o-mini	Coarse credit assignment (outcome-only)
Most principled long-term approach	Non-reproducible (web content changes)

Pattern E: Self-Critique / Self-RAG

Pros	Cons
Elegant adaptive retrieval	Requires full model training
On-demand retrieval (not always-retrieve)	Older base model (Llama-2)
Configurable at inference time	High training cost
ICLR 2024 Oral (top 1%)	Implicit confidence (beam weights, not explicit scores)
Preserves LLM versatility when retrieval not needed	Static corpus

Pattern F: Adversarial Multi-Agent

Pros	Cons
Most rigorous verification	Very expensive (multiple expensive model calls per claim)
Model independence prevents correlated errors	Complex orchestration
Principled falsification methodology	Slow (sequential claim processing)
Rejected claims preserved (not deleted)	Tiny community (<20 stars)
Epistemic layer tracking	Requires careful prompt engineering for each role

Pattern G: Verify-then-Retrieve Loop

Pros	Cons
Verification drives retrieval (causally correct)	Few existing implementations to reference
Natural per-claim confidence output	Convergence criteria need tuning
Separates concerns (generate ≠ verify)	Multiple model calls per claim per iteration
Handles all verdict types (supported/contested/insufficient)	Risk of infinite loops without proper termination
Evidence stays verbatim (not summarized)	Cold-start problem (first-pass claims may all be weak)

---

5. State-of-the-Art Findings (June 2026)

Critical Research Findings

1. **Citation quality ≠ factual accuracy** — "Cited but Not Verified" (arXiv 2605.06635) shows best models achieve >94% valid links but only 39-77% factual accuracy. More retrieval does NOT equal better grounding – accuracy drops ~42% as tool calls scale from 2 to 150.

2. **Self-evaluation provides zero calibration information** without external labels — "The Verification Tax" (arXiv 2604.12951) proves this formally. Implications: you MUST calibrate against ground truth, not self-assess.

3. **Best systems achieve only 19.9% on structural verification** — "Intent to Evidence" (arXiv 2603.25342) shows agents struggle with "long-horizon synthesis and intersection-heavy verification."

4. **Faithfulness ≠ correctness** — VERITAS (arXiv 2510.13272) shows outcome-correct agents can still reason unfaithfully. Step-level faithfulness rewards are needed beyond outcome rewards.

5. **Diverse judges >> homogeneous panels** — Diverse medium-capacity models (Gemini-Pro + Mixtral + PaLM 2-M) achieve 91% on GSM-8K, surpassing GPT-4 alone. But 9 correlated judges provide only ~2 effective independent votes (arXiv 2605.29800).

6. **Search Intensity Scaling is emergent** — DeepDiver (arXiv 2505.24332, NeurIPS 2025 Spotlight) shows 7B models autonomously escalate search frequency on harder problems when trained with RL. This validates adaptive retrieval as a learnable behavior.

7. **Sequential scaling >> parallel scaling** for reasoning — "Let Me Think" (arXiv 2505.21825, NeurIPS 2025) proves theoretically that long sequential reasoning chains offer exponential advantages over breadth-first parallel sampling for certain tasks.

8. **Verification is the bottleneck for test-time scaling** — "Large Language Monkeys" (arXiv 2407.21787) shows coverage scales log-linearly with samples, but selection methods plateau without automatic verifiers.

Performance Benchmarks (Top Systems)

System	BrowseComp	GAIA	FRAMES	Notes
MiroThinker H1	74.0%	82.7%	-	300-600 tool calls per task
Alibaba DeepResearch-30B	SOTA	-	SOTA	Fine-tuned MoE, 3.3B active
DeepDive-32B	SOTA (open)	-	-	KG-augmented training
Search-R1 (Qwen2.5-7B)	-	-	+41% over RAG	RL-trained
WebThinker (QwQ-32B)	-	-	-	Outperforms Grok3 DeeperSearch

---

6. Gaps in the Current Landscape

These represent **opportunities for differentiation** – areas where no existing system provides a strong solution:

Gap	Description	Who's Closest
Combined verification + confidence + adaptive retrieval	No system unifies all three in one loop	Self-RAG (2/3), your DRA (3/3)
Calibrated confidence (ECE, isotonic regression)	Almost no system provides calibrated per-claim confidence	None deployed; only academic papers
Verbatim evidence preservation	Most systems summarize evidence before verification	MARs (partial)
Model independence enforcement	Few systems explicitly enforce distinct model families for generator vs. verifier	MARs only
Statistical evaluation rigor	No open-source system uses Cohen's kappa / McNemar / Holm for evaluation	None
Verification-driven retrieval	Retrieval triggered by verification failure (not by generation confidence)	ADORE (closest), your DRA
Claim-level granularity with epistemic status	Tracking supported/contested/insufficient per atomic claim	MARs (admission gate), your DRA

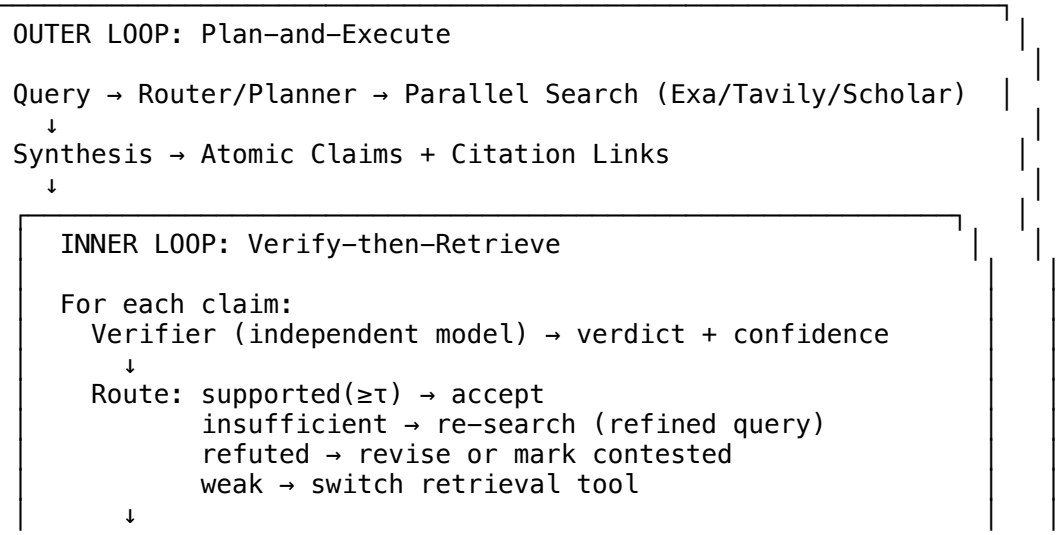
---

7. Recommendations

7.1 Recommended Architecture: Hybrid Pattern (B + G + elements of F)

For the Deep Research Agent MVP, I recommend combining:

1. **Plan-and-Execute (Pattern B)** for the outer loop — decompose query, parallel search, synthesize
2. **Verify-then-Retrieve Loop (Pattern G)** as the inner verification loop — the core differentiator
3. **Adversarial elements (Pattern F)** for the verifier design — different model family from generator



```
│  │    Loop until convergence                                    │  │
│  └──────────────────────────────────────────────────────────────┘  │
│    ↓                                                               │
│  Calibrate confidence (temperature scaling / isotonic)             │
│    ↓                                                               │
│  Generate Report (supported + contested + insufficient claims)     │
└────────────────────────────────────────────────────────────────────┘
```

7.2 Recommended Tech Stack

Component	Choice	Rationale
Orchestration	LangGraph	Dominant ecosystem; graph-based = natural for branching/looping; studio for debugging
Generator LLM	AWS Bedrock (Claude Sonnet)	Your existing infra; strong long-form quality
Verifier LLM	AWS Bedrock (different family: e.g., Llama 3 or Mistral via Bedrock)	Model independence; same API surface
Judge Panel	3 models from distinct families via Bedrock	Diversity maximizes effective independent votes
Primary Search	Exa (semantic) + Tavily (web)	Exa for conceptual search; Tavily for breadth; route by query type
Academic Search	Semantic Scholar API	Free; purpose-built for papers
Confidence Calibration	Isotonic regression on held-out set	Validated by Verification Tax paper; simple and effective
Evaluation	Custom harness (ECE + Cohen's kappa + McNemar)	No existing tool does this; build it
Frontend/CLI	CLI first (Python Typer) → FastAPI + Streamlit for demo	Matches MVP timeline
Deployment	Docker + AWS Bedrock (no GPU needed for inference)	Accessible; no training infrastructure needed

7.3 Key Design Decisions & Rationale

Decision	Recommendation	Literature Support
Single-agent vs. multi-agent?	Hybrid: single orchestrator + specialized tools (not full multi-agent)	Multi-agent adds debugging complexity without proportional quality gain for research tasks (GPT-Researcher finding)
Verification approach?	Independent LLM-judge (different model family from generator)	Diversity of Thought paper; Nine Judges paper; avoid correlated errors
When to retrieve more?	Verification verdict drives retrieval (not generation confidence)	DeepDiver SIS; FLARE; causally correct
Confidence representation?	Raw score → isotonic regression → calibrated probability	Verification Tax proves self-eval = zero info; must calibrate externally
Evidence handling?	Verbatim passages only; never summarize before verification	"Cited but Not Verified" shows summarization breaks grounding
Claim granularity?	Atomic claims (one verifiable fact per claim)	Intent-to-Evidence paper; CHARM (cascading hallucination)

Decision	Recommendation	Literature Support
Report format?	Claim ledger with epistemic status (supported/contested/insufficient)	Novel; no existing system does this well
Ablation choice?	Verifier A (LLM-judge) vs. Verifier B (cross-encoder); measure ECE before/after calibration	Isolates verification method AND calibration impact
Convergence criteria?	τ threshold + max_iters + no-progress detection + budget cap	Standard in RL literature; prevents infinite loops
Cost management?	Cheap models for planning/search, expensive for synthesis/verification only	LangChain Open DR pattern; validated cost-quality trade-off

7.4 What NOT to Do

Based on landscape analysis, avoid these common pitfalls:

1. **Don't conflate multi-source aggregation with verification** — Having 20+ sources (GPT-Researcher approach) does NOT verify claims. Source count ≠ grounding.
2. **Don't use the same model family for generation and verification** — Correlated errors make this theater, not verification.
3. **Don't summarize evidence before checking claims against it** — Summarization loses the grounding signal.
4. **Don't rely on self-evaluation for confidence** — Formally proven to provide zero calibration information.
5. **Don't over-engineer the multi-agent coordination** — AutoGen is deprecated; even Microsoft moved away. Simple orchestration (LangGraph) with clear role separation beats complex agent negotiation.
6. **Don't train a model unless you have 8xA100+** — RL-trained approaches (Search-R1) are SOTA but inaccessible for MVP. Prompt-based verification with calibration is the pragmatic choice.
7. **Don't skip the evaluation harness** — Most repos have no rigorous evaluation. This is your differentiator – "spine = evaluation" is architecturally novel.

7.5 Priority Order for Implementation

Given the ~2 day timeline in the assessment:

Priority	Component	Why
P0	Claim extraction + independent verification loop	Core differentiator; proves the thesis
P0	Calibrated confidence (before/after comparison)	Required ablation; demonstrates rigor
P1	Adaptive retrieval (verdict → re-search)	Completes the loop; shows it works E2E
P1	Structured report with claim ledger	Required output format; separates claims from evidence

Priority	Component	Why
P2	Multi-tool search routing	Nice-to-have; use Exa or Tavily alone for MVP
P2	Judge panel (3+ models)	Defer to evaluation phase; single judge sufficient for MVP
P3	Full evaluation harness (ECE, kappa, McNemar)	Include in design doc; implement subset for MVP

---

8. Source Index

Top GitHub Repositories (by relevance)

#	Repository	Stars	Key Contribution
1	stanford-oval/storm	28.3K	Multi-perspective research (NAACL 2024)
2	assafelovic/gpt-researcher	27.6K	Production plan-execute-publish pipeline
3	Alibaba-NLP/DeepResearch	19.4K	RL-trained MoE research model
4	dzhng/deep-research	19.1K	Minimal recursive exploration (<500 LoC)
5	langchain-ai/open_deep_research	11.7K	Official LangGraph reference implementation
6	MiroMindAI/MiroThinker	8.3K	Interactive scaling, SOTA BrowseComp
7	zilliztech/deep-searcher	7.9K	Agentic RAG for private data
8	jina-ai/node-DeepResearch	5.2K	Iterative search-read-reason with token budget
9	PeterGriffinJin/Search-R1	4.9K	RL framework for search-augmented reasoning
10	SkyworkAI/DeepResearchAgent	3.4K	Self-evolution protocol (RSPL + SEPL)
11	AkariAsai/self-rag	2.4K	Adaptive retrieval with reflection tokens (ICLR 2024)
12	RUC-NLPIR/WebThinker	1.5K	Single-generation in-thinking tools (NeurIPS 2025)
13	Tencent/CognitiveKernel-Pro	517	Hierarchical multi-agent with inference-time verification
14	FredFang1216/MARs	18	Adversarial Builder/Skeptic/Arbiter with claim DAG
15	tarun7r/deep-research-agent	172	LangGraph with credibility scoring

Key Arxiv Papers (by theme)

Agent Architectures: - Search-R1 (2503.09516) — RL for interleaved search and reasoning - DeepResearch-9K (2603.01152) — Benchmark + training framework - DeepDiver (2505.24332) — Search Intensity Scaling (NeurIPS 2025 Spotlight) - WebThinker — Single-generation in-thinking tools (NeurIPS 2025) - STORM (2402.14207) — Multi-perspective research (NAACL 2024) - Co-STORM (2408.15232) — Collaborative research (EMNLP 2024)

Verification & Grounding: - Veri-R1 (2510.01932) — RL-trained claim verification with search - VERITAS (2510.13272) — Faithfulness rewards beyond correctness (TMLR) - Cited but Not Verified (2605.06635) — Source attribution evaluation - Intent to Evidence (2603.25342) — Category-theoretic structural evaluation -

CHARM (2606.04435) — Cascading hallucination in agentic RAG - PROClaim (2603.28488) — Courtroom-style adversarial verification

Multi-Agent & Debate: - Multi-agent Debate (2305.14325) — Foundational debate paper (MIT) - Diversity of Thought (2410.12853) — Diverse models > homogeneous - Nine Judges (2605.29800) — Correlated errors limit panel effectiveness - SAGE (2603.15255) — Multi-agent self-evolution - MAD-Fact (2510.22967) — Debate for long-form factuality

Test-Time Compute: - Scaling Test-Time Compute (2408.03314) — Foundational theory (Google DeepMind) - Large Language Monkeys (2407.21787) — Repeated sampling scaling - Let Me Think (2505.21825) — Sequential > parallel (NeurIPS 2025) - Kimi k1.5 (2501.12599) — Long context RL scaling

Adaptive Retrieval: - Self-RAG (2310.11511) — Reflection tokens (ICLR 2024 Oral) - FLARE (2305.06983) — Active retrieval (EMNLP 2023) - AlignRAG (2504.14858) — Critic Language Model (NeurIPS 2025) - ADORE (2601.18267) — Enterprise iterative retrieval-reflection

Evaluation: - MT-Bench / Chatbot Arena (2306.05685) — LLM-as-judge (NeurIPS 2023) - The Verification Tax (2604.12951) — Fundamental limits of self-evaluation - Verdict (2502.18018) — Judge-time compute scaling library - FLAMe (2407.10817) — Foundational autorater models

Process Rewards: - PRA (2604.09482) — Process rewards for frozen policies (ICML 2026) - TreePS-RAG (2601.06922) — Monte Carlo tree process supervision - HiPRAG (2510.07794) — Hierarchical rewards (ICLR 2026) - SLATE (2602.23440) — Truncated step-level sampling

Self-Correction: - SCoRe (2409.12917) — Multi-turn RL self-correction (Google DeepMind) - RAAT (2405.20978) — Adaptive adversarial training for noise robustness (ACL 2024)

Key YouTube Resources

- **LangChain Open Deep Research (Official Build)** — Architecture walkthrough with benchmarks
- **GPT-Researcher Creator Talk (Assaf Elovic)** — Production multi-agent patterns
- **Self-reflective RAG with LangGraph** — CRAG/Self-RAG implementation patterns
- **Stanford STORM Demos** — Multi-perspective research approach
- **NVIDIA Deep Research Agent Course** — Enterprise deployment patterns
- **AWS Strands 5-Agent Framework** — Bedrock-native research agents

---

Appendix: Your DRA Design Validation

Based on the landscape analysis, your current design choices are **well-validated by the literature**:

Your Design Choice	Validation Source	Confidence
Generator ≠ Verifier (model independence)	Diversity of Thought (2410.12853); Nine Judges (2605.29800)	Strong
Per-claim verification, not end-to-end	CHARM (2606.04435); Intent-to-Evidence (2603.25342)	Strong
Calibrated confidence (not raw scores)	Verification Tax (2604.12951)	Strong (formal proof)
Adaptive retrieval triggered by verification	DeepDiver (2505.24332); FLARE (2305.06983); Self-RAG	Strong
Process rewards > outcome-only	VERITAS (2510.13272); HiPRAG; TreePS-RAG	Strong

Your Design Choice	Validation Source	Confidence
Evidence is verbatim, never summarized	Cited but Not Verified (2605.06635)	Strong
DeBERTa cross-encoder as Verifier B	Standard NLI approach; fast + interpretable	Moderate
LangGraph orchestration	Dominant ecosystem; graph-native for loops	Strong (pragmatic)
Exa/Tavily/Semantic Scholar for retrieval	Community standard; free tiers available	Strong

Novel elements with no direct precedent (your differentiators): - Combined verification + calibrated confidence + adaptive retrieval in one loop - ECE metric applied to deployed research agent confidence - Isotonic regression for confidence recalibration - Full statistical evaluation harness (Cohen's kappa + McNemar + Holm) - Explicit "claim ledger" with supported/contested/insufficient epistemic states

---

End of report.